QUEUE DATA STRUCTURE

GATE DA – Python + Data Structures

Complete Theory + Properties + Numericals + PYQs + Python

GATE Important

GATE DA Focus:
For Queue, concentrate especially on:

1. FIFO principle
2. Enqueue and Dequeue
3. Front and Rear
4. Operation tracing
5. Circular Queue
6. Deque
7. Priority Queue
8. Time complexity
9. Python implementation
10. Queue applications, especially BFS

1 1. What is a Queue?

Definition

A **Queue** is a linear data structure in which insertion takes place at one end called the **Rear** and deletion takes place at the other end called the **Front**.

The queue follows:

$$\boxed{\text{FIFO} = \text{First In First Out}}$$

The element inserted first is removed first.

Example

Suppose we insert:

$$10,\ 20,\ 30,\ 40$$

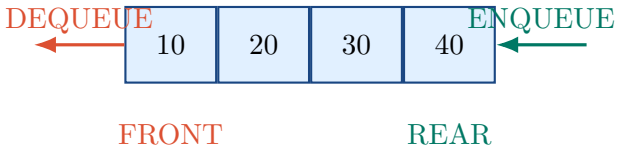
Then:

$$\text{Front} \rightarrow 10 \mid 20 \mid 30 \mid 40 \leftarrow \text{Rear}$$

The first element removed will be:

$$\boxed{10}$$

2 2. Queue Diagram



GATE Important

$$\boxed{\text{DEQUEUE} \rightarrow \text{FRONT}}$$

$$\boxed{\text{ENQUEUE} \rightarrow \text{REAR}}$$

Remember this for GATE operation-tracing questions.

3 3. Basic Terminology

Term	Meaning
Queue	Linear FIFO data structure
Front	Position from which deletion occurs
Rear	Position where insertion occurs
Enqueue	Insert an element
Dequeue	Remove an element
Peek / Front	View first element without removing it
Overflow	Insertion attempted when queue is full
Underflow	Deletion attempted when queue is empty

4 4. Fundamental Operations

4.1 4.1 Enqueue

Definition

Enqueue means inserting an element at the rear of the queue.

If:

$$Q = [10, 20, 30]$$

then:

$$\text{enqueue}(40)$$

gives:

$$Q = [10, 20, 30, 40]$$

4.2 4.2 Dequeue

Definition

Dequeue means removing the element from the front.

If:

$$Q = [10, 20, 30, 40]$$

then:

$$\text{dequeue}()$$

returns:

10

and:

$$Q = [20, 30, 40]$$

4.3 4.3 Peek

Peek returns the front element without removing it.

$$Q = [10, 20, 30]$$

$$\text{peek}() = 10$$

Queue remains:

$$[10, 20, 30]$$

5 5. Queue ADT

The Abstract Data Type of Queue can be represented as:

$$\boxed{Q = \langle \text{enqueue}, \text{dequeue}, \text{peek}, \text{isEmpty}, \text{isFull} \rangle}$$

Operation	Purpose
enqueue(x)	Insert x at rear
dequeue()	Remove front element
peek()	Return front element
isEmpty()	Check whether queue is empty
isFull()	Check whether queue is full

6 6. Queue State Representation

Suppose:

$$Q = [10, 20, 30, 40]$$

Then:

$$front = 0$$

$$rear = 3$$

for a zero-indexed array implementation.

GATE Important

For an array of size n:

$$0 \le front \le n - 1$$

and:

$$0 \le rear \le n - 1$$

for a simple non-circular queue.

7 7. Linear Queue Using Array

Suppose array size is 5.

10	20	30	40	50
0	1	2	3	4

Here:

$$front = 0, \qquad rear = 4$$

The queue is full.
If we dequeue twice:

$$10,\ 20$$

then:

X	X	30	40	50
0	1	2	3	4

Now there are two unused locations at the beginning.
However, a simple linear queue may still consider the queue full because:

$$rear = 4 = n - 1$$

This is called:

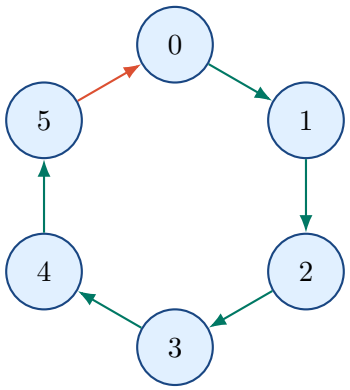
$$\boxed{\text{False Overflow}}$$

8 8. Circular Queue

Definition

A **Circular Queue** treats the last position of an array as connected to the first position.

Conceptually:



The next position can be calculated using:

$$\boxed{next = (current + 1) \bmod n}$$

9 9. Circular Queue Important Formulas

For queue size n:

$$\boxed{rear = (rear + 1) \bmod n}$$

during enqueue.
During dequeue:

$$\boxed{front = (front + 1) \bmod n}$$

A commonly used convention with one empty slot is:

$$\boxed{(rear + 1) \bmod n = front}$$

for full condition.
Empty condition:

$$\boxed{front = rear}$$

under that convention.

Common Trap

Do not mix circular queue conventions.
Some implementations use:

$$front = -1, \quad rear = -1$$

while others use an always-valid front/rear pointer and reserve one slot.
Always inspect the convention given in the question.

10 10. Circular Queue Example

Suppose:

$$n = 5$$

and:

$$rear = 4$$

After another enqueue:

$$rear = (4 + 1) \bmod 5$$

$$rear = 0$$

Therefore:

$$\boxed{rear = 0}$$

The rear wraps around.

Example

If:

$$front = 3, \qquad rear = 4, \qquad n = 5$$

then after one enqueue:

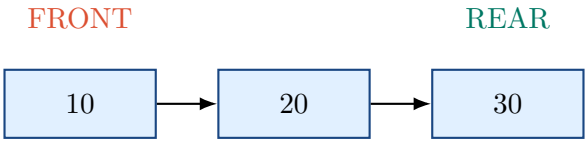
$$rear = (4 + 1) \bmod 5 = 0$$

After another enqueue:

$$rear = (0 + 1) \bmod 5 = 1$$

11 11. Linked List Implementation

A queue can also be implemented using a linked list.



For efficient operations:

$$\boxed{enqueue = O(1)}$$

$$\boxed{dequeue = O(1)}$$

provided both front and rear pointers are maintained.

12 12. Queue Using Python List

A simple implementation is:

```
queue = []

queue.append(10)
queue.append(20)
queue.append(30)

print(queue)

x = queue.pop(0)

print("Removed:", x)
print(queue)
```

Output:

```
[10, 20, 30]
Removed: 10
[20, 30]
```

Common Trap

Although Python's list supports `pop(0)`, it is **not efficient** for a queue because elements have to be shifted.

$$\boxed{\texttt{list.pop(0)} = O(n)}$$

13 13. Python Queue Using collections.deque

For practical Python programming:

```
from collections import deque

q = deque()

q.append(10)
q.append(20)
q.append(30)

print(q)

x = q.popleft()

print(x)
print(q)
```

Complexities:

$$\boxed{\texttt{append()} = O(1)}$$

$$\boxed{\texttt{popleft()} = O(1)}$$

Therefore:

$$\boxed{\texttt{deque} \text{ is preferred for FIFO queues}}$$

14 14. Complete Queue Class in Python

```
class Queue:

    def __init__(self):
        self.items = []

    def is_empty(self):
        return len(self.items) == 0

    def enqueue(self, x):
        self.items.append(x)

    def dequeue(self):
        if self.is_empty():
            return "Underflow"
        return self.items.pop(0)

    def front(self):
        if self.is_empty():
            return "Queue is empty"
        return self.items[0]

    def size(self):
        return len(self.items)


q = Queue()

q.enqueue(10)
q.enqueue(20)
```

```
q.enqueue(30)

print(q.front())
print(q.dequeue())
print(q.dequeue())
```

15 15. Complexity Comparison

Implementation	Enqueue	Dequeue	Peek
Array with rear/front	$O(1)$	$O(1)$	$O(1)$
Linked List	$O(1)$	$O(1)$	$O(1)$
Python list + append/pop(0)	$O(1)$	$O(n)$	$O(1)$
Python deque	$O(1)$	$O(1)$	$O(1)$

GATE Important

GATE Memory Rule

`list.append()` $\rightarrow O(1)$ amortized
`list.pop(0)` $\rightarrow O(n)$
`deque.append()` $\rightarrow O(1)$
`deque.popleft()` $\rightarrow O(1)$

16 16. Queue Overflow and Underflow

16.1 Overflow

Overflow occurs when an insertion is attempted in a full queue.

Full Queue + Enqueue $\Rightarrow$ Overflow

16.2 Underflow

Underflow occurs when deletion is attempted in an empty queue.

Empty Queue + Dequeue $\Rightarrow$ Underflow

17 17. Queue Operation Tracing

This is one of the most important GATE skills.

Suppose:

$$Q = []$$

Operations:

$$enqueue(5)$$

$$enqueue(10)$$

$$enqueue(15)$$

$$dequeue()$$

$$enqueue(20)$$

$$dequeue()$$

Trace:

Operation	Queue
Initial	[]
enqueue(5)	[5]
enqueue(10)	[5, 10]
enqueue(15)	[5, 10, 15]
dequeue()	[10, 15]
enqueue(20)	[10, 15, 20]
dequeue()	[15, 20]

The removed elements are:

$$5,\ 10$$

18 18. Important Queue Property

Suppose elements are inserted in this order:

$$A, B, C, D, E$$

Then the dequeue order must be:

$$\boxed{A, B, C, D, E}$$

provided no other operation changes the sequence.

GATE Important

A normal queue cannot remove D before A, B, and C.
This is the defining FIFO property.

19 19. Queue vs Stack

Property	Queue	Stack
Principle	FIFO	LIFO
Insertion	Rear	Top
Deletion	Front	Top
Insertion operation	Enqueue	Push
Deletion operation	Dequeue	Pop
Typical application	BFS	DFS

$$\boxed{\text{Queue} \rightarrow FIFO}$$

$$\boxed{\text{Stack} \rightarrow LIFO}$$

20 20. Deque

Definition

A **Deque** (Double-Ended Queue) is a data structure in which insertion and deletion can occur at both ends.

Operations:

$$insertFirst()$$

$$insertLast()$$

$$removeFirst()$$

$$removeLast()$$

Diagram:



GATE Important

Queue: insertion at rear, deletion at front

Deque: insertion/deletion at both ends

21 21. Priority Queue

Definition

A Priority Queue removes elements according to their priority rather than strictly according to insertion time.

For example:

$$(A, 3),\ (B, 1),\ (C, 2)$$

If lower number means higher priority:

$$B \to C \to A$$

Common Trap

Do not confuse a normal queue with a priority queue.
Normal queue:

FIFO

Priority queue:

Priority determines removal

22 22. Python Priority Queue

Using `heapq`:

```python
import heapq

pq = []

heapq.heappush(pq, (3, "A"))
heapq.heappush(pq, (1, "B"))
heapq.heappush(pq, (2, "C"))

print(heapq.heappop(pq))
```

Output:

```
(1, 'B')
```

23 23. Queue and BFS

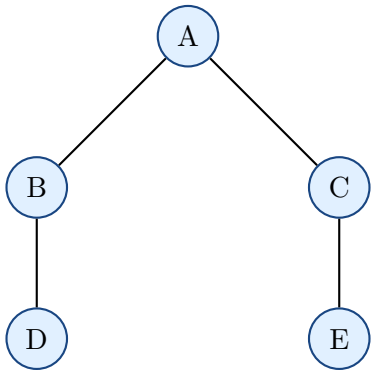
Breadth First Search uses a queue.

BFS → Queue

Basic idea:

1. Start from source.
2. Enqueue source.
3. Dequeue one vertex.
4. Visit its unvisited neighbours.
5. Enqueue those neighbours.
6. Repeat.

Example graph:



Starting from A:

$$Q = [A]$$

Dequeue A:

$$Q = []$$

Enqueue B,C:

$$Q = [B, C]$$

Dequeue B:

$$Q = [C]$$

Enqueue D:

$$Q = [C, D]$$

Therefore BFS begins:

$$\boxed{A, B, C, D, E}$$

24 24. Important BFS–Queue Connection

Important Property

BFS explores vertices in increasing order of their shortest-path distance from the source in an unweighted graph.
The queue guarantees that vertices discovered earlier are processed earlier.

For an unweighted graph:

$$\boxed{\text{BFS time complexity} = O(V + E)}$$

with adjacency-list representation.

25 25. Numerical 1 – Basic Queue

Example

Initially:

$$Q = []$$

Perform:

$$enqueue(10)$$

$$enqueue(20)$$

$$dequeue()$$

$$enqueue(30)$$

$$dequeue()$$

Find the removed elements.

Solution:

$$[] \to [10] \to [10, 20]$$

First dequeue:

$$10$$

Queue:

$$[20]$$

Then:

$$[20, 30]$$

Second dequeue:

$$20$$

Therefore:

$$\boxed{10,\ 20}$$

26 26. Numerical 2 – Count Operations

Suppose a queue initially contains 25 elements.
Then:

$$12 \text{ enqueue operations}$$

and:

$$8 \text{ dequeue operations}$$

are performed.
Final number of elements:

$$25 + 12 - 8$$

$$\boxed{29}$$

provided no overflow occurs.

27 27. Numerical 3 – Circular Queue

Suppose:

$$n = 8$$

and:

$$rear = 7$$

After one enqueue:

$$rear = (7 + 1) \bmod 8$$

$$\boxed{rear = 0}$$

After another:

$$rear = (0 + 1) \bmod 8$$

$$\boxed{rear = 1}$$

28 28. Numerical 4 – Circular Queue

Suppose:

$$n = 10, \quad rear = 8$$

Three enqueue operations occur.
First:

$$rear = (8 + 1) \bmod 10 = 9$$

Second:

$$rear = (9 + 1) \bmod 10 = 0$$

Third:

$$rear = (0 + 1) \bmod 10 = 1$$

Therefore:

$$\boxed{rear = 1}$$

29 29. Numerical 5 – Queue Size

Initially:

$$|Q| = 17$$

Operations:

$$7 \text{ enqueue}$$

$$11 \text{ dequeue}$$

Therefore:

$$|Q| = 17 + 7 - 11$$

$$\boxed{|Q| = 13}$$

30 30. Important Formula Sheet

GATE Important

Queue Core Formulas

$$\boxed{\text{FIFO} = \text{First In First Out}}$$

$$\boxed{\text{Enqueue} \rightarrow \text{Rear}}$$

$$\boxed{\text{Dequeue} \rightarrow \text{Front}}$$

$$\boxed{\text{Peek} \rightarrow \text{Front}}$$

For circular queue:

$$\boxed{next = (current + 1) \bmod n}$$

$$\boxed{rear = (rear + 1) \bmod n}$$

$$\boxed{front = (front + 1) \bmod n}$$

For queue size:

$$\boxed{\text{Final size} = \text{Initial size} + \text{Enqueues} - \text{Dequeues}}$$

31 31. GATE Important Results

Concept	Result
Queue principle	FIFO
Stack principle	LIFO
Enqueue end	Rear
Dequeue end	Front
Peek	Front element
BFS uses	Queue
DFS commonly uses	Stack / recursion
Circular increment	$(i + 1) \bmod n$
Python list dequeue	$O(n)$
Python deque dequeue	$O(1)$
Linked-list enqueue	$O(1)$ with rear pointer
Linked-list dequeue	$O(1)$ with front pointer

32 32. REAL GATE DA PYQ – 2024

REAL GATE DA PYQ

GATE DA 2024 – Queue / Deque
The fundamental operations in a double-ended queue D are:

$$insertFirst(e)$$

Insert a new element e at the beginning.

$$insertLast(e)$$

Insert a new element e at the end.

$$removeFirst()$$

Remove and return the first element.

$$removeLast()$$

Remove and return the last element.
In an empty deque, the following operations are performed:

$$\begin{aligned}
&insertFirst(10)\\
&insertLast(32)\\
&a \leftarrow removeFirst()\\
&insertLast(28)\\
&insertLast(17)\\
&a \leftarrow removeFirst()\\
&a \leftarrow removeLast()
\end{aligned}$$

Find the final value of a.

Solution

Trace the queue:

$$[]$$

After:

$$insertFirst(10)$$

$$[10]$$

After:

$$insertLast(32)$$

$$[10, 32]$$

After:

$$removeFirst()$$

$$a = 10$$

$$[32]$$

After:

$$insertLast(28)$$

$$[32, 28]$$

After:

$$insertLast(17)$$

$$[32, 28, 17]$$

After:

$$removeFirst()$$

$$a = 32$$

$$[28, 17]$$

Finally:

$$removeLast()$$

Therefore:

$$\boxed{a = 17}$$

33 33. REAL GATE DA PYQ – FIFO/LIFO

REAL GATE DA PYQ

GATE DA 2024
Match the following:

Column 1	Column 2
(p) First In First Out	(i) Stacks
(q) Lookup Operation	(ii) Queues
(r) Last In First Out	(iii) Hash Tables

Which matching is correct?

(A) (p)–(ii), (q)–(iii), (r)–(i)

(B) (p)–(ii), (q)–(i), (r)–(iii)

(C) (p)–(i), (q)–(ii), (r)–(iii)

(D) (p)–(i), (q)–(iii), (r)–(ii)

Solution

FIFO corresponds to Queue:

$$p \rightarrow ii$$

Lookup is associated with Hash Tables:

$$q \rightarrow iii$$

LIFO corresponds to Stack:

$$r \rightarrow i$$

Therefore:

$$\boxed{\text{Option A}}$$

34 34. GATE-Style Numerical Question 1

REAL GATE DA PYQ

A queue initially contains:

$$[5, 10, 15, 20]$$

Perform:

$$dequeue()$$

$$enqueue(25)$$

$$dequeue()$$

$$enqueue(30)$$

What is the final queue?

Solution

Initial:

$$[5, 10, 15, 20]$$

After dequeue:

$$[10, 15, 20]$$

After enqueue(25):

$$[10, 15, 20, 25]$$

After dequeue:

$$[15, 20, 25]$$

After enqueue(30):

$$\boxed{[15, 20, 25, 30]}$$

35 35. GATE-Style Numerical Question 2

REAL GATE DA PYQ

A circular queue has size 7.
Initially:

$$rear = 5$$

Four enqueue operations are performed.
Find the final value of $rear$.

Solution

First:

$$(5+1) \bmod 7 = 6$$

Second:

$$(6+1) \bmod 7 = 0$$

Third:

$$(0+1) \bmod 7 = 1$$

Fourth:

$$(1+1) \bmod 7 = 2$$

Hence:

$$\boxed{rear = 2}$$

36 36. GATE-Style Numerical Question 3

REAL GATE DA PYQ

A queue contains 40 elements.
During an algorithm:

$$\text{15 elements are inserted}$$

and:

$$\text{22 elements are removed.}$$

How many elements remain?

Solution

$$40 + 15 - 22 = 33$$

Therefore:

$$\boxed{33}$$

37 37. GATE-Style MCQ

REAL GATE DA PYQ

Which Python data structure is most appropriate for efficient FIFO operations?

(A) list with `pop(0)`

(B) tuple

(C) `collections.deque`

(D) set

Solution

The preferred choice is:

$$\boxed{\texttt{collections.deque}}$$

because insertion/removal at the ends is efficient.
Answer:

$$\boxed{\text{C}}$$

38 38. Common GATE Traps

Common Trap

Trap 1
Do not write:

$$\text{Queue} = LIFO$$

Correct:

$$\boxed{\text{Queue} = FIFO}$$

Trap 2
Enqueue is NOT performed at front.

$$\boxed{\text{Enqueue} \rightarrow \text{Rear}}$$

Trap 3
Dequeue is NOT performed at rear.

$$\boxed{\text{Dequeue} \rightarrow \text{Front}}$$

Trap 4
Python:

$$\texttt{pop(0)}$$

is not $O(1)$.

$$\boxed{O(n)}$$

Trap 5
Deque is not identical to a normal queue.
Deque allows operations at both ends.

39 39. Queue Applications

Important applications:

- BFS
- CPU scheduling
- Printer scheduling
- Network packet buffering
- Task scheduling
- Producer-consumer systems
- Resource management
- Request processing

- Simulation
- Breadth-first traversal

40 40. Types of Queues

Type	Main Idea
Simple Queue	FIFO
Circular Queue	Last position connects to first
Deque	Insertion/deletion at both ends
Priority Queue	Removal based on priority
Input-restricted Deque	Insertion restricted to one end
Output-restricted Deque	Deletion restricted to one end

41 41. Input-Restricted Deque

Insertion is allowed only at one end.

Deletion can occur at both ends.

$$\boxed{\text{Insertion: one end}}$$

$$\boxed{\text{Deletion: both ends}}$$

42 42. Output-Restricted Deque

Deletion is allowed only at one end.

Insertion can occur at both ends.

$$\boxed{\text{Insertion: both ends}}$$

$$\boxed{\text{Deletion: one end}}$$

43 43. Queue and Memory

For an array queue:

$$\text{Space} = O(n)$$

For a linked-list queue containing n nodes:

$$\text{Space} = O(n)$$

Therefore:

$$\boxed{\text{Queue space complexity} = O(n)}$$

44 44. Advanced Numerical – Circular Queue

Example

A circular queue has capacity:

$$n = 6$$

Suppose:

$$front = 4, \qquad rear = 5$$

Perform two enqueue operations.
First:

$$rear = (5 + 1) \bmod 6 = 0$$

Second:

$$rear = (0 + 1) \bmod 6 = 1$$

Therefore:

$$\boxed{rear = 1}$$

The rear wraps around the array.

45 45. Advanced Numerical – Operation Count

Example

A queue initially has 18 elements.
Operations:

- 13 enqueue operations
- 9 dequeue operations
- 4 enqueue operations
- 7 dequeue operations

Final size:

$$18 + 13 - 9 + 4 - 7$$

$$= 19$$

Therefore:

$$\boxed{19}$$

46 46. Queue Pseudocode

46.1 Enqueue

```
ENQUEUE(Q, x)
```

```
if Q is full:
    OVERFLOW
else:
    rear = rear + 1
    Q[rear] = x
```

46.2 Dequeue

```
DEQUEUE(Q)

if Q is empty:
    UNDERFLOW
else:
    x = Q[front]
    front = front + 1
    return x
```

47 47. Circular Queue Pseudocode

```
ENQUEUE(x):

if queue is full:
    OVERFLOW

rear = (rear + 1) mod n
Q[rear] = x

DEQUEUE():

if queue is empty:
    UNDERFLOW

x = Q[front]
front = (front + 1) mod n
return x
```

48 48. Important Comparison

Structure	FIFO	LIFO	Both Ends	Priority
Queue	Yes	No	No	No
Stack	No	Yes	No	No
Deque	No	No	Yes	No
Priority Queue	No	No	Depends	Yes

49 49. Practice Questions

1. A queue initially contains $[2, 4, 6, 8]$. After two dequeue operations, what remains?
2. A queue contains 10 elements. Five are inserted and three are removed. Find the final size.
3. For a circular queue of size 10, if $rear = 9$, what is the next rear position?
4. Which data structure is used by BFS?
5. What is the time complexity of `list.pop(0)`?

6. What is the time complexity of `deque.popleft()`?
7. Which queue allows insertion and deletion at both ends?
8. What is FIFO?
9. If elements $1, 2, 3, 4, 5$ are inserted into a normal queue, in what order are they removed?
10. A circular queue has $n = 8$ and $rear = 6$. Three enqueue operations are performed. Find the final rear index.

50 50. Practice Answers

Question	Answer
1	$[6, 8]$
2	12
3	0
4	Queue
5	$O(n)$
6	$O(1)$
7	Deque
8	First In First Out
9	$1, 2, 3, 4, 5$
10	1

51 51. GATE DA Queue Revision Sheet

GATE Important

Must Remember

- FIFO
- Enqueue → Rear
- Dequeue → Front
- Peek → Front
- BFS → Queue
- Circular queue
- Modulo operation
- Deque
- Priority Queue
- Overflow
- Underflow
- Python deque
- Operation tracing
- Complexity

$$\begin{aligned} \text{Queue} &\Rightarrow FIFO \\ \text{Stack} &\Rightarrow LIFO \\ \text{BFS} &\Rightarrow Queue \\ \text{DFS} &\Rightarrow Stack \\ \text{Circular next} &\Rightarrow (i+1) \bmod n \end{aligned}$$

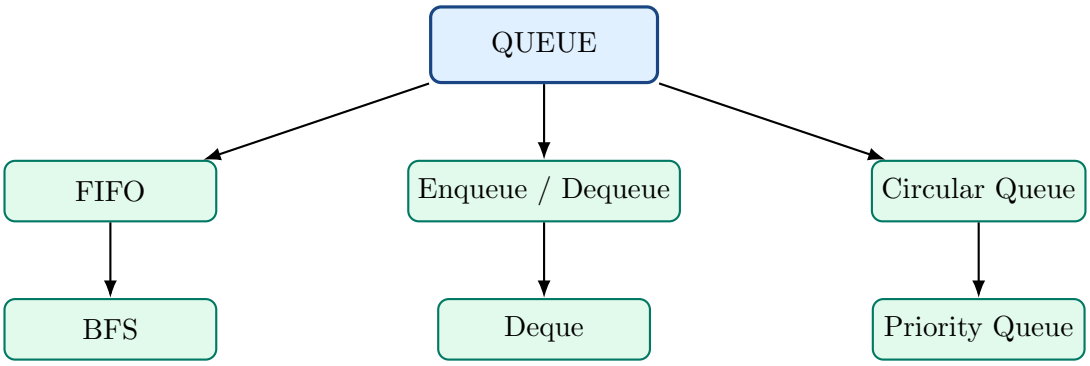
52 52. Final GATE Checklist

Before moving to the next DSA topic, you should be able to answer all of the following without looking at notes:

1. What is FIFO?

2. What is enqueue?
3. What is dequeue?
4. What are front and rear?
5. What is queue overflow?
6. What is queue underflow?
7. What is false overflow?
8. Why is circular queue required?
9. How does modulo work in circular queue?
10. What is a deque?
11. What is a priority queue?
12. What is the difference between queue and stack?
13. Why does BFS use a queue?
14. Why is `list.pop(0)` $O(n)$?
15. Why is `deque.popleft()` $O(1)$?
16. Can you trace 10–15 queue operations quickly?
17. Can you solve circular queue numerical questions?
18. Can you identify FIFO/LIFO from a matching question?

GATE DA MASTER MEMORY MAP



GATE DA DSA – QUEUE

Concept → Trace → Complexity → Python → PYQ